

IT PROFESSIONAL PORTFOLIO

Cloud Engineering and Security Projects

PROJECT 44

Expose an S3 Bucket with a Misconfigured Bucket Policy

Amazon S3 | Bucket Policies | Public Access | Resource Policies

AUTHOR	Michael Salazar
ENVIRONMENT	Personal AWS Lab
VERSION	v1.0.0
RELEASE DATE	August 3, 2026
STATUS	Complete - Follow-Up Detection Pending

Executive Summary

Created a second controlled public Amazon S3 bucket in **TestAccount1**, this time using a **bucket policy** rather than Access Control Lists. Public access was allowed during bucket creation, and the bucket policy was edited to grant **s3:ListBucket** and **s3:GetObject** through two exposure patterns: an unrestricted **Principal: "*"** statement and an **aws:SourceIp = 0.0.0.0/0** condition. Both patterns make the bucket accessible to the public Internet.

SECURITY FINDING

The two policy statements are effectively redundant. Principal "*" already allows any principal, while 0.0.0.0/0 represents all IPv4 addresses. They are included together because both are common misconfiguration patterns that defenders should recognize.

Business Problem

S3 bucket policies are resource-based policies that can grant access to principals outside the AWS account. That flexibility supports legitimate public content and cross-account sharing, but a broad principal, an overly permissive IP condition, or the wrong actions can expose data without requiring an IAM policy on the external identity.

Objectives

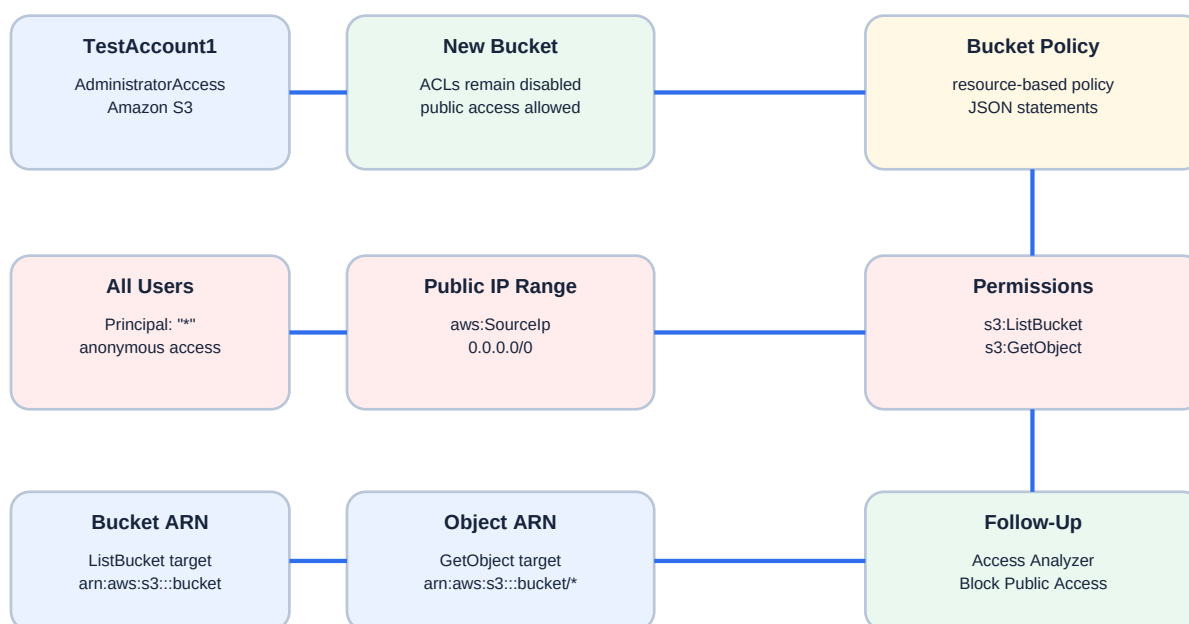
- * Sign into TestAccount1 with AdministratorAccess.
- * Create a new general purpose S3 bucket.
- * Keep ACLs disabled and allow public access for the controlled lab.
- * Open the bucket policy editor.
- * Grant s3:ListBucket and s3:GetObject to Principal "*".
- * Add a second statement using aws:SourceIp 0.0.0.0/0.
- * Include both the bucket ARN and object ARN in the resource list.
- * Create a public-bucket target for upcoming Access Analyzer and Block Public Access labs.

Environment

Cloud Provider	Amazon Web Services
Account	TestAccount1 / AdministratorAccess
Primary Service	Amazon S3
Access Mechanism	S3 resource-based bucket policy
ACL Configuration	ACLs not required for this lab; bucket policy provides public access

Public Principal	Principal: "*"
Public Network Condition	aws:SourceIp: 0.0.0.0/0
Bucket-Level Permission	s3:ListBucket
Object-Level Permission	s3:GetObject
Status	Intentionally public; follow-up detection and remediation pending

Architecture



Both statements expose the same bucket publicly; 0.0.0.0/0 is effectively the entire Internet.

Figure 1 - Public S3 exposure through two bucket-policy patterns.

Implementation Summary

1. Logged into TestAccount1 with AdministratorAccess.
2. Opened Amazon S3 and created a new bucket.
3. Allowed public access during bucket creation.
4. Opened the bucket Permissions tab and edited the bucket policy.
5. Added PublicReadViaAllUsers with Effect Allow and Principal "*".
6. Granted s3:GetObject and s3:ListBucket.

7. Added PublicReadViaIPAddress using aws:SourceIp 0.0.0.0/0.
8. Included the bucket ARN for ListBucket and the bucket object ARN ending in /* for GetObject.
9. Saved the policy to create a deliberately public bucket-policy configuration.

Policy Pattern

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadViaAllUsers",
      "Effect": "Allow",
      "Principal": "*",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3::example-public-bucket/*",
        "arn:aws:s3::example-public-bucket"
      ]
    },
    {
      "Sid": "PublicReadViaIPAddress",
      "Effect": "Allow",
      "Principal": "*",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3::example-public-bucket/*",
        "arn:aws:s3::example-public-bucket"
      ],
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "0.0.0.0/0"
        }
      }
    }
  ]
}
```

VERIFY ARN

Evidence note: the screenshot displays the placeholder ARN example-public-bucket. Before saving a real bucket policy, the resource values must be replaced with that bucket's actual ARN while retaining both the bucket ARN and the /* object ARN.

Action and Resource Mapping

s3:ListBucket	arn:aws:s3::bucket-name	Allows enumeration of object keys in the bucket.
s3:GetObject	arn:aws:s3::bucket-name/*	Allows objects to be downloaded when their keys are known.
Principal: "*"	All principals	Allows anonymous and external access unless an explicit deny blocks it.
aws:SourceIp 0.0.0.0/0	All IPv4 addresses	Does not meaningfully restrict access; it represents the public Internet.

Evidence

Figure 2 - Misconfigured Public S3 Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadViaAllUsers",
      "Effect": "Allow",
      "Principal": "*",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::example-public-bucket/*",
        "arn:aws:s3:::example-public-bucket"
      ]
    },
    {
      "Sid": "PublicReadViaIPAddress",
      "Effect": "Allow",
      "Principal": "*",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::example-public-bucket/*",
        "arn:aws:s3:::example-public-bucket"
      ],
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "0.0.0.0/0"
        }
      }
    }
  ]
}
```

The supplied policy evidence shows two Allow statements. `PublicReadViaAllUsers` uses Principal `"*"`. `PublicReadViaIPAddress` also uses Principal `"*"` with `aws:SourceIp` set to `0.0.0.0/0`. Both statements grant `s3:GetObject` and `s3:ListBucket` and include bucket and object resource patterns.

Security Analysis

Bucket policies are evaluated as resource-based policies. An external principal can receive access directly from the bucket policy when no explicit deny blocks it.

Combining `ListBucket` and `GetObject` is especially risky because an unauthenticated user can enumerate object names and then retrieve the objects.

Principal `"*"` is a direct public-access pattern. It should only be used for a deliberately public resource with narrowly scoped actions and data.

The IP condition `0.0.0.0/0` is not a restriction. It includes every IPv4 address and therefore creates Internet-wide access.

The bucket ARN and object ARN serve different purposes. `ListBucket` applies to the bucket itself, while `GetObject` applies to objects under the `/` path.

An explicit deny from Block Public Access, an SCP, a permissions boundary, or another applicable policy can still prevent the public access.

Lessons Learned

Bucket policies are JSON resource policies and are the preferred modern mechanism for most S3 access control. Resource policies can grant access to identities that have no corresponding S3 permission in their own IAM policy. Public access can be introduced through a wildcard principal or an ineffective network condition. ListBucket and GetObject must be mapped to the correct bucket and object ARNs. A syntactically valid policy can still be dangerously over-permissive. Detection and preventive controls are necessary because policy reviews alone do not scale.

Production Improvements

- * Keep S3 Block Public Access enabled at the account and bucket levels.
- * Enable IAM Access Analyzer to detect public and cross-account S3 access.
- * Use explicit principals, organization IDs, VPC endpoint conditions, or narrow IP ranges instead of wildcards.
- * Add an explicit deny for requests that do not use HTTPS.
- * Separate ListBucket and GetObject into clearly scoped statements.
- * Use policy validation and Infrastructure as Code scanning before deployment.
- * Apply SCPs or organization policies to prevent unauthorized public bucket policies.
- * Keep controlled lab buckets empty and remove or remediate them after validation.

Skills Demonstrated

Bucket policies	Resource-based policies	Wildcard principals
ListBucket / GetObject	Bucket and object ARNs	IP conditions
Public exposure testing	Access Analyzer preparation	Block Public Access planning

Resume Bullet

RESUME

Built a controlled S3 bucket-policy exposure lab by granting public ListBucket and GetObject access through wildcard-principal and 0.0.0.0/0 policy statements, then documented the action/resource mappings, misconfiguration risks, and planned Access Analyzer and Block Public Access remediation.

STAR Interview Story

Situation: The prior lab demonstrated public S3 exposure using legacy ACLs. The next objective was to examine the preferred modern access mechanism and how it can also be misconfigured.

Task: Create a deliberately public S3 bucket using a resource-based bucket policy and document the dangerous policy patterns.

Action: I created a new bucket, allowed public access, added one statement with Principal "*" and another using aws:SourceIp 0.0.0.0/0, and granted ListBucket and GetObject against the bucket and object ARNs.

Result: The lab demonstrated two common bucket-policy exposure patterns and produced a target for the upcoming Access Analyzer and Block Public Access detection and prevention labs.

Version History

v1.0.0	August 3, 2026	Initial documentation. Created a controlled public S3 bucket through wildcard-principal and 0.0.0.0/0 bucket-policy statements, documented ListBucket/GetObject resource mapping, embedded the supplied policy evidence, and identified the placeholder ARN verification requirement.

References

- * Cloud Security Lab a Week - Accidentally Expose All Your Stuff on S3 with a Bucket Policy.
- * User-provided misconfigured bucket-policy evidence.
- * Project 43 - Expose an S3 Bucket with ACLs for Public Access Testing.